EGB242 Assignment 2, Section 1 %%

Table of Contents

Initialise workspace 1
Section 1.1 plot audioMultiplexNoisy in time and frequency domain 1
Section 1.2 with function block 2
1.2 Plotting 3
Section 1.3 6
Section 1.4 7
1.4 Plotting 7
Section 1.5 10
Function definitions 12

This file is a template for your MATLAB solution to Section 1.

Before starting to write code, generate your data with the ??? as described in the assignment task.

Initialise workspace

```
clear all; close all;
load DataA2 audioMultiplexNoisy fs sid;

% Begin writing your MATLAB solution below this line.R
load('DataA2.mat')
```

Section 1.1 plot audioMultiplexNoisy in time and frequency domain

```
%Create a time vector
samples = length(audioMultiplexNoisy);
T = samples/fs;
timeVector = linspace(0, T, samples + 1);
timeVector(end) = [];

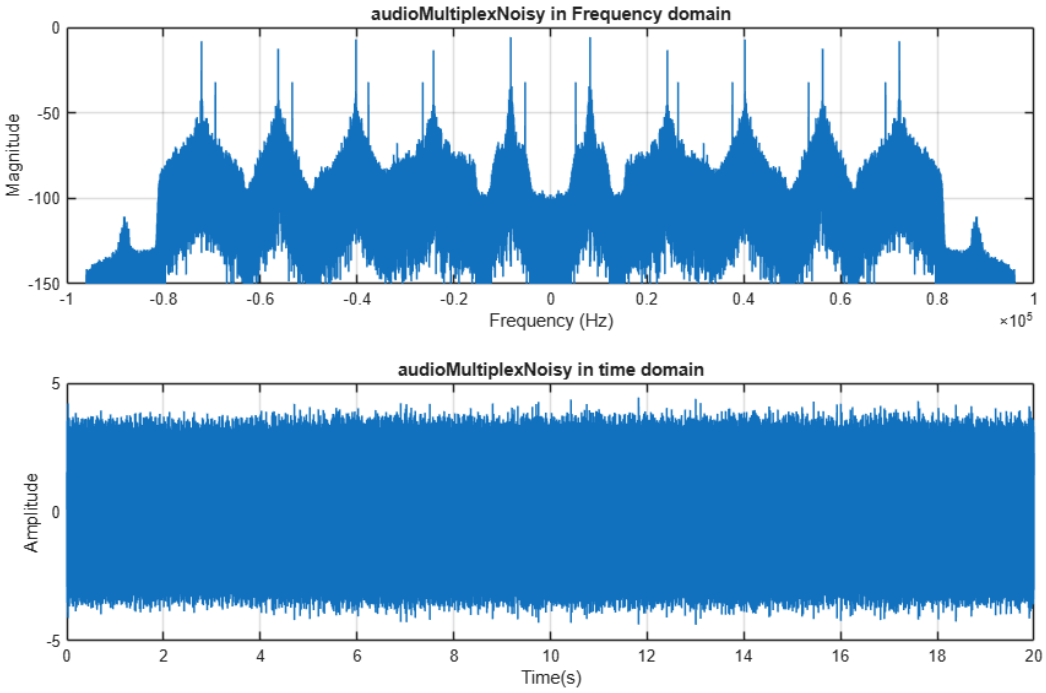
% Create a frequency vector
fA = linspace(-fs/2, fs/2, samples + 1);
fA(end) = [];

% Convert to the frequency domain
AudioMultiplexNoisy_f = normalised_fft(audioMultiplexNoisy);

% Plotting in time domain
figure;
subplot(2,1,2);
plot(timeVector, audioMultiplexNoisy);
title('audioMultiplexNoisy in time domain');
xlabel('Time(s)');
ylabel('Amplitude');
```

```
grid on;

% Plot in the frequency domain
subplot(2,1,1);
plot(fA, amp2dB(abs(fftshift(AudioMultiplexNoisy_f))));
title('audioMultiplexNoisy in Frequency domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
ylim([-150 0])
grid on;
```



Section 1.2 with function block

```
%Set the LPF cutoff frequency for each carrier frequency
cutoffFreq1 = 4700;
cutoffFreq2 = 8600;
cutoffFreq3 = 8600;
cutoffFreq4 = 6500;
cutoffFreq5 = 8500;

% Define carrier frequency
CF1 = 8210;
CF2 = 24190;
CF3 = 40210;
CF4 = 56290;
CF5 = 72160;

% Function calls for each carrier frequency
```

```
[C1, demod1, audioreceived1, audioReceivedFreq1] = ...
    carrierFreq(CF1, timeVector, audioMultiplexNoisy, cutoffFreq1, fs);
[C2, demod2, audioreceived2, audioReceivedFreq2] = ...
    carrierFreq(CF2, timeVector, audioMultiplexNoisy, cutoffFreq2, fs);
[C3, demod3, audioreceived3, audioReceivedFreq3] = ...
    carrierFreq(CF3, timeVector, audioMultiplexNoisy, cutoffFreq3, fs);
[C4, demod4, audioreceived4, audioReceivedFreq4] = ...
    carrierFreq(CF4, timeVector, audioMultiplexNoisy, cutoffFreq4, fs);
[C5, demod5, audioreceived5, audioReceivedFreq5] = ...
    carrierFreq(CF5, timeVector, audioMultiplexNoisy, cutoffFreq5, fs);
```

1.2 Plotting

Make an array of plotting values

```
plotFreq = {audioReceivedFreq1, audioReceivedFreq2, audioReceivedFreq3, ...
    audioReceivedFreq4, audioReceivedFreq5};

plotTime = {audioreceived1, audioreceived2, audioreceived3, ...
    audioreceived4, audioreceived5};

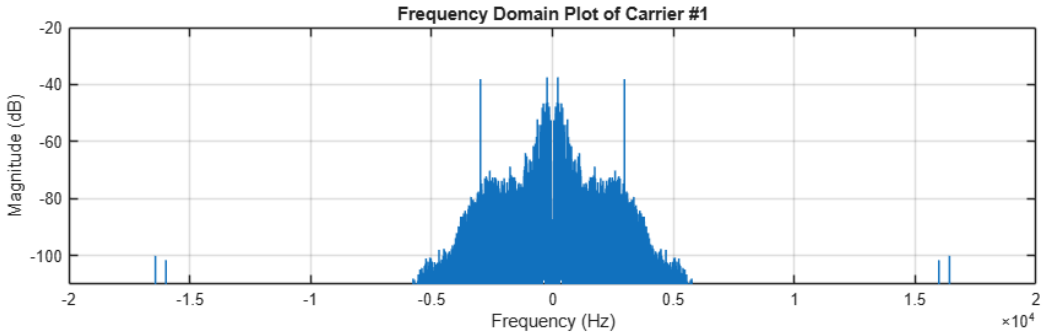
plotCutoff = {cutoffFreq1, cutoffFreq2, cutoffFreq3, cutoffFreq4, ...
    cutoffFreq5};

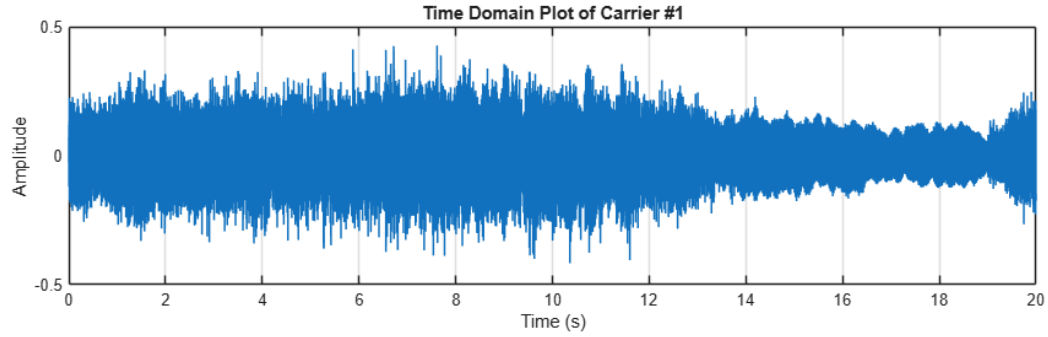
for i = 1:5

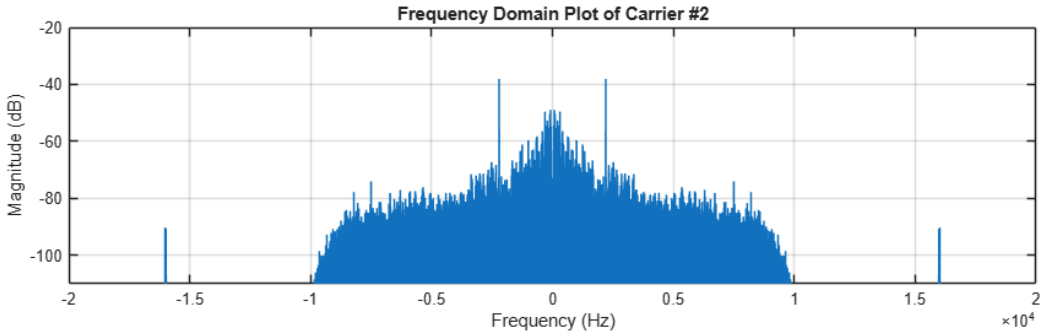
    figure;
    subplot(2, 1, 1);
    plot(fA, amp2dB(abs(fftshift(plotFreq{i}))));
    title(['Frequency Domain Plot of Carrier #' num2str(i)]);
    xlabel('Frequency (Hz)');
    ylabel('Magnitude (dB)');
    xlim([-2e4, 2e4]);
    ylim([-110 -20])
    grid on;

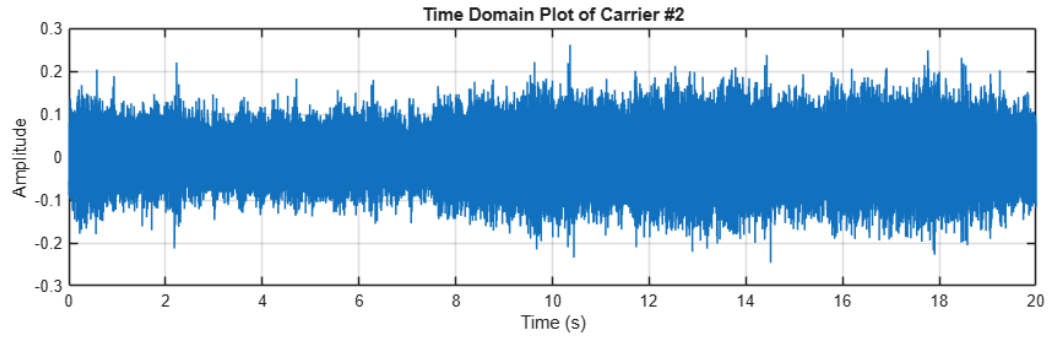
    subplot(2, 1, 2);
    plot(timeVector, plotTime{i});
    title(['Time Domain Plot of Carrier #' num2str(i)]);
    xlabel('Time (s)');
    ylabel('Amplitude');
    grid on;

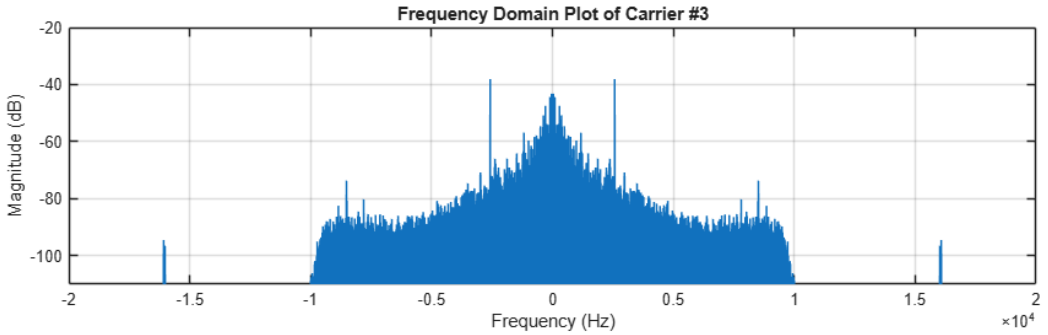
end
```

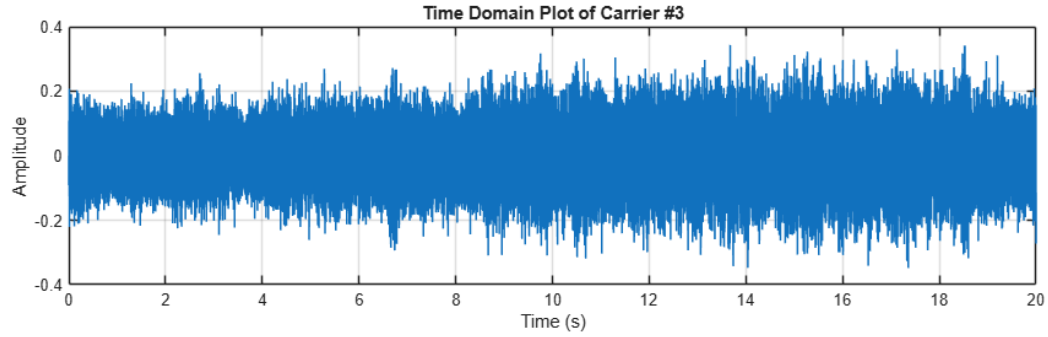

Frequency Domain Plot of Carrier #1
Magnitude (dB)
-20
-40
-60
-80
-100
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2
Frequency (Hz)
×10^4

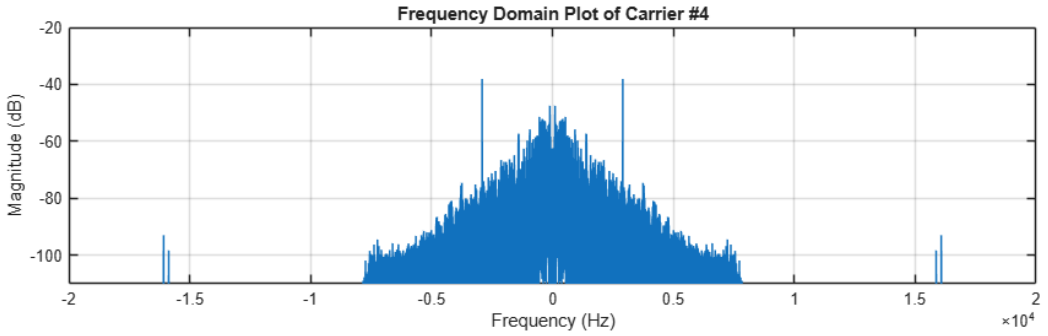

Time Domain Plot of Carrier #1
Amplitude
0.5
0
-0.5
0
2
4
6
8
10
12
14
16
18
20
Time (s)

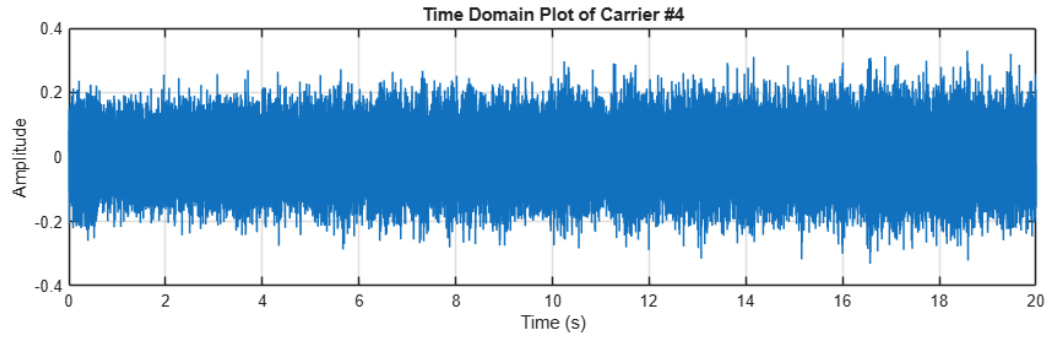

Frequency Domain Plot of Carrier #2
Magnitude (dB)
-20
-40
-60
-80
-100
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2
Frequency (Hz)
×10^4

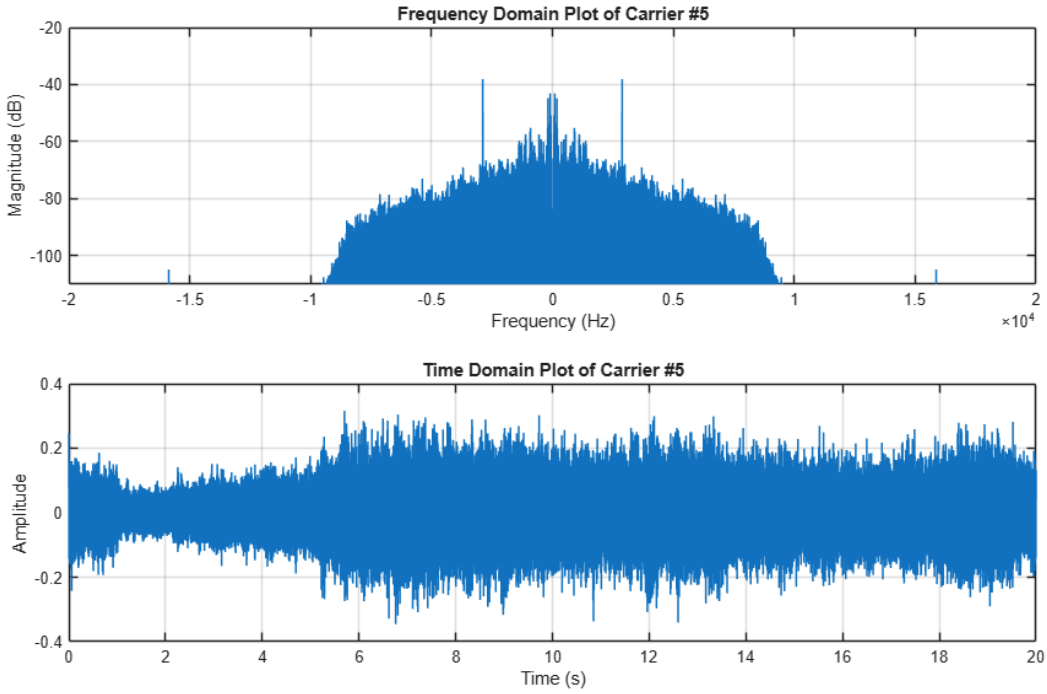

Time Domain Plot of Carrier #2
Amplitude
0.3
0.2
0.1
0
-0.1
-0.2
-0.3
0
2
4
6
8
10
12
14
16
18
20
Time (s)


Frequency Domain Plot of Carrier #3
Magnitude (dB)
-20
-40
-60
-80
-100
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2
Frequency (Hz)
×10^4


Time Domain Plot of Carrier #3
Amplitude
0.4
0.2
0
-0.2
-0.4
0
2
4
6
8
10
12
14
16
18
20
Time (s)


Frequency Domain Plot of Carrier #4
Magnitude (dB)
-20
-40
-60
-80
-100
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2
Frequency (Hz)
×10^4


Time Domain Plot of Carrier #4
Amplitude
0.4
0.2
0
-0.2
-0.4
0
2
4
6
8
10
12
14
16
18
20
Time (s)



Section 1.3

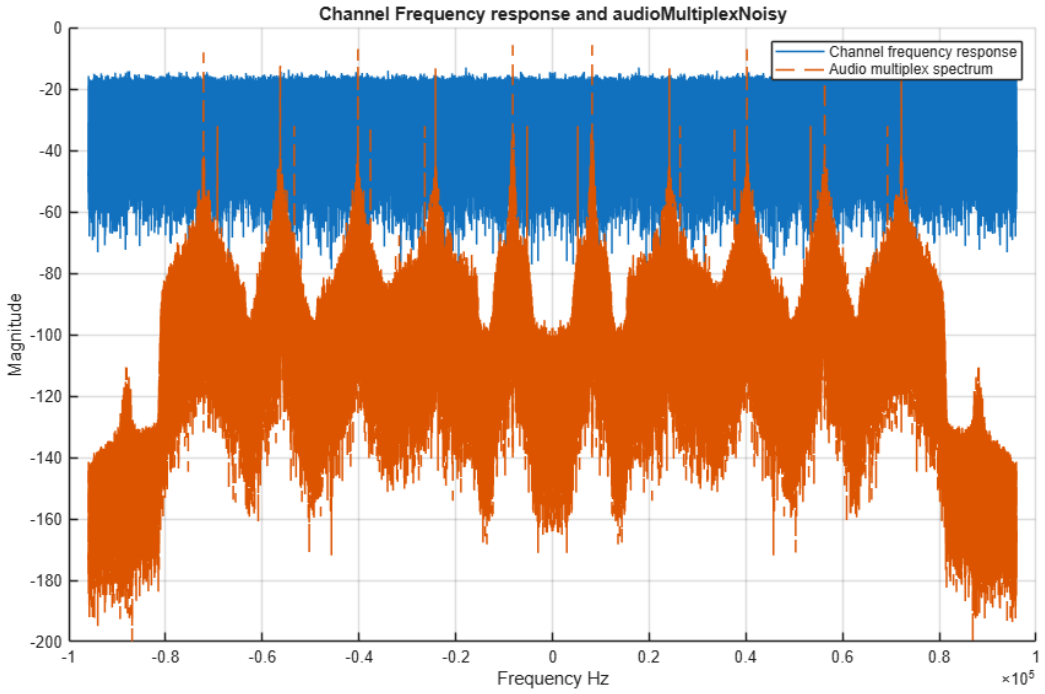
Create an impulse

```
impulseAudio = [fs, zeros(1, samples - 1)];

%Use the channel feature to model the impulse response
channelIR_t = channel(sid, impulseAudio, fs);

%Find the frequency response and create the frequency vector
ChannelIR_f = normalised_fft(channelIR_t);


%Plotting both the magnitude spectrum and multiplexed audio on the same
% graph
figure;
hold on;
plot(fA, amp2dB(abs(fftshift(ChannelIR_f))));
plot(fA, amp2dB(abs(fftshift(AudioMultiplexNoisy_f))), '--');
title('Channel Frequency response and audioMultiplexNoisy');
xlabel('Frequency Hz');
ylabel('Magnitude');
legend('Channel frequency response', 'Audio multiplex spectrum');
%ylim([0, 0.2]);
%xlim([0, 0.05]);
grid on;
hold off;
```



Section 1.4

Set the length of audioMultiplexNoisy

```
samples =  length(audioMultiplexNoisy);

% Reverse channel distortion
revdistAudio = AudioMultiplexNoisy_f ./ ChannelIR_f;
audioequalised_t = ifft(revdistAudio) * fs;

% Demodulate the individual signals into cleanAudio

[C1, demod1, audioclean1_t, audioclean1_f] = ...
    carrierFreq(CF1, timeVector, audioequalised_t, cutoffFreq1, fs);
[C2, demod2, audioclean2_t, audioclean2_f] = ...
    carrierFreq(CF2, timeVector, audioequalised_t, cutoffFreq2, fs);
[C3, demod3, audioclean3_t, audioclean3_f] = ...
    carrierFreq(CF3, timeVector, audioequalised_t, cutoffFreq3, fs);
[C4, demod4, audioclean4_t, audioclean4_f] = ...
    carrierFreq(CF4, timeVector, audioequalised_t, cutoffFreq4, fs);
[C5, demod5, audioclean5_t, audioclean5_f] = ...
    carrierFreq(CF5, timeVector, audioequalised_t, cutoffFreq5, fs);
```

1.4 Plotting

Make an array of plotting values

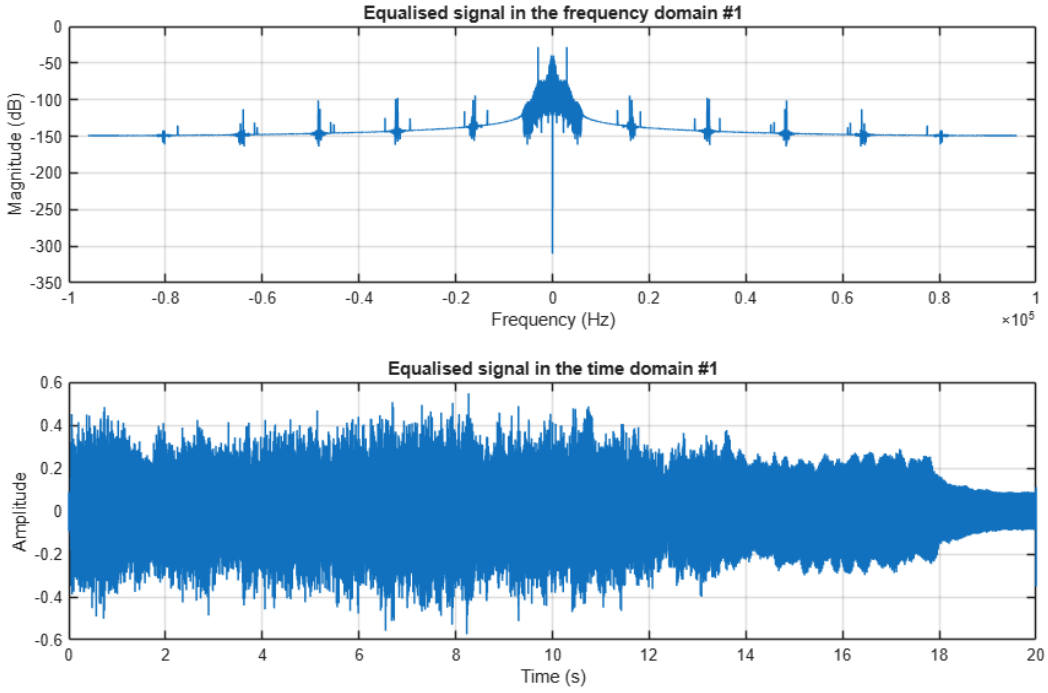
```
plot1_4 = {audioclean1_t, audioclean2_t, audioclean3_t, ...
    audioclean4_t, audioclean5_t};
```

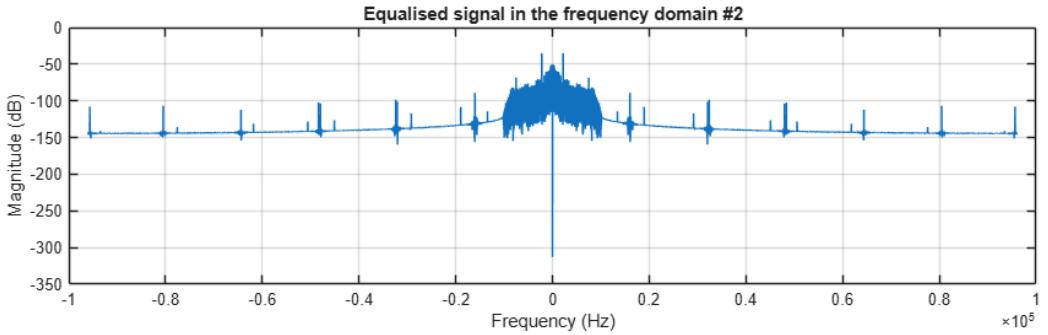
```
for i = 1:5

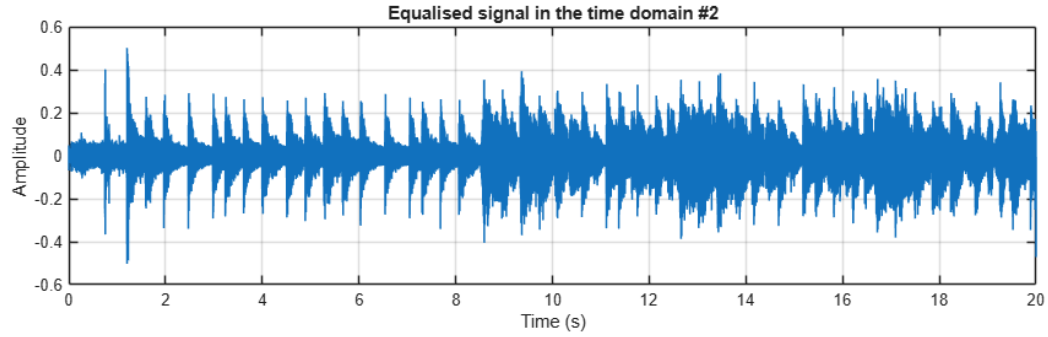
    figure;
    subplot(2, 1, 1);
    plot(fA, amp2dB(abs(fftshift(fft(plot1_4{i}) / samples))));
    title(['Equalised signal in the frequency domain #' num2str(i)]);
    xlabel('Frequency (Hz)');
    ylabel('Magnitude (dB)');
    grid on;

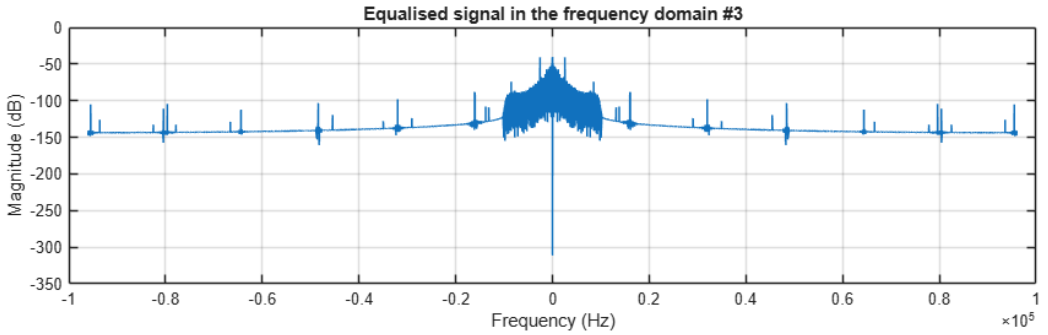
    subplot(2, 1, 2);
    plot(timeVector, plot1_4{i});
    title(['Equalised signal in the time domain #' num2str(i)]);
    xlabel('Time (s)');
    ylabel('Amplitude');
    grid on;

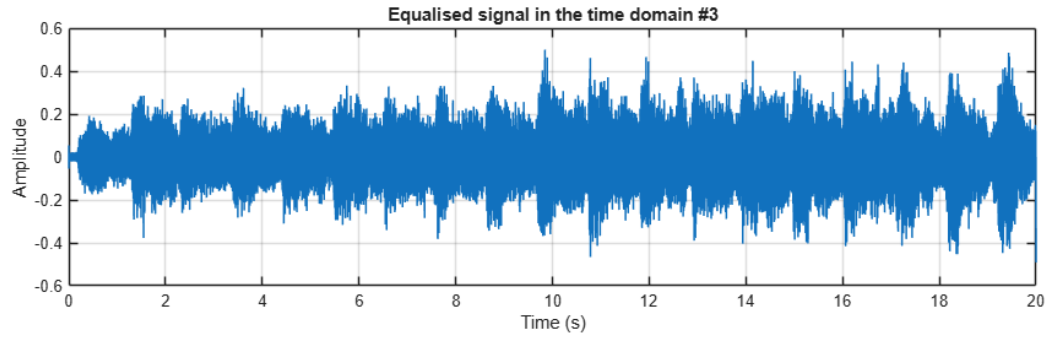
end
```

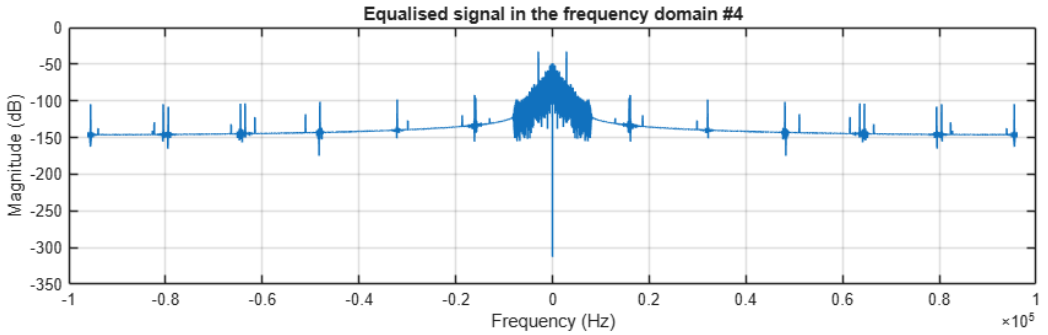


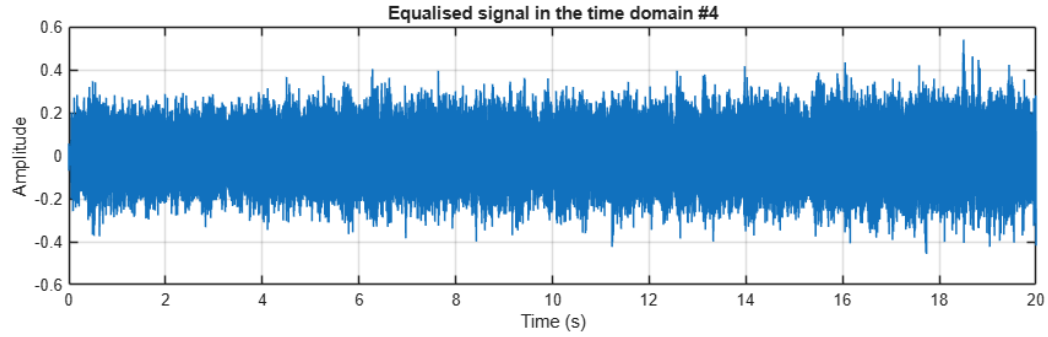

Equalised signal in the frequency domain #2
Magnitude (dB)
Frequency (Hz)
×10^5

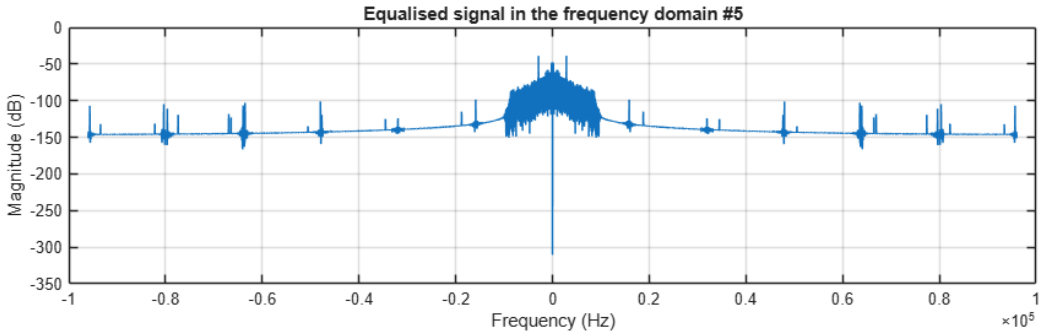

Equalised signal in the time domain #2
Amplitude
Time (s)

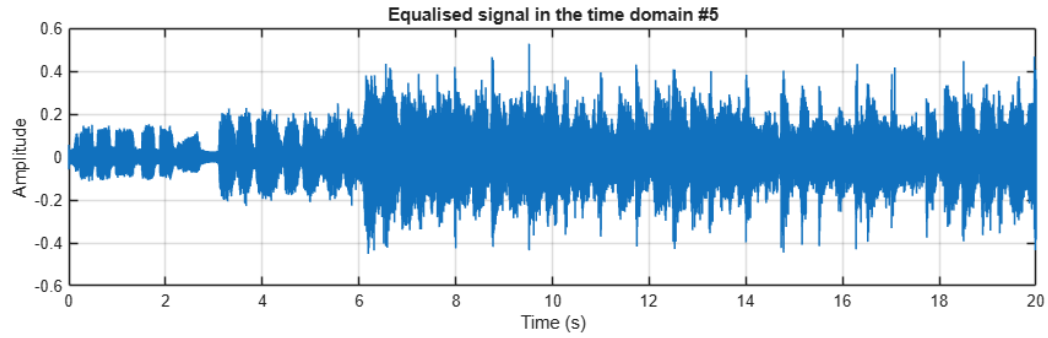

Equalised signal in the frequency domain #3
Magnitude (dB)
Frequency (Hz)
×10^5


Equalised signal in the time domain #3
Amplitude
Time (s)









Section 1.5

```
%plot1_5 = {audioclean1_f, audioclean2_f, audioclean3_f, audioclean4_f, ...
    %audioclean5_f};
```

```
%for i = 1:5

    % Plot the frequency domain
    %figure()
    %subplot(2,1,1) % Used to test.  May not be needed.
    %plot(fA, abs(fftshift(plot1_5{i})));
    %title(['Equalised signal magnitude spectrum #' num2str(i)]);
    %xlabel('Frequency (Hz)');
    %ylabel('Magnitude');
    %grid on;

    % Plot the time domain
    %subplot(2,1,2) % Used to test.  May not be needed.
    %plot(fA, amp2dB(abs(fftshift(plot1_5{i}))));
    %title(['Equalised signal magnitude spectrum #' num2str(i)]);
    %xlabel('Frequency (Hz)');
    %ylabel('Magnitude (dB)');
    %grid on;

%end

% Frequency with which we search above for single-tone components
initial_f = 2000;

[cleaned_signal_t1, Cleaned_Signal_f1] = ...
    removetones(audioclean1_f, fA, initial_f, cutoffFreq1, fs);
[cleaned_signal_t2, Cleaned_Signal_f2] = ...
    removetones(audioclean2_f, fA, initial_f, cutoffFreq2, fs);
[cleaned_signal_t3, Cleaned_Signal_f3] = ...
    removetones(audioclean3_f, fA, initial_f, cutoffFreq3, fs);
[cleaned_signal_t4, Cleaned_Signal_f4] = ...
    removetones(audioclean4_f, fA, initial_f, cutoffFreq4, fs);
[cleaned_signal_t5, Cleaned_Signal_f5] = ...
    removetones(audioclean5_f, fA, initial_f, cutoffFreq5, fs);

plot1_5 = {cleaned_signal_t1, cleaned_signal_t2, cleaned_signal_t3, ...
    cleaned_signal_t4, cleaned_signal_t5};

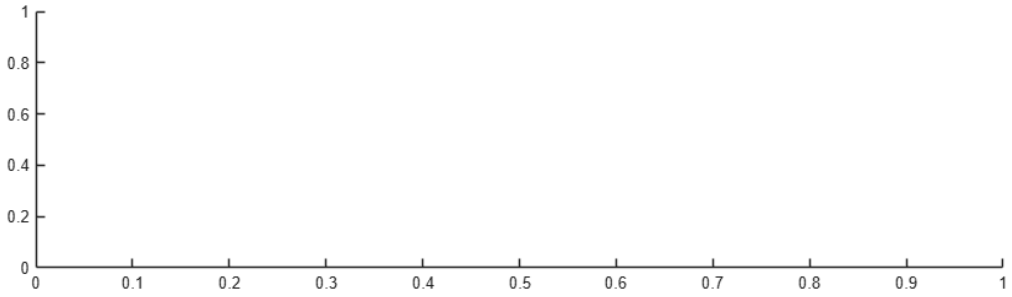
plot1_6 = {Cleaned_Signal_f1, Cleaned_Signal_f2, Cleaned_Signal_f3, ...
    Cleaned_Signal_f4, Cleaned_Signal_f5};

for i = 1:5

    % Plot the final clean audio signal for each 5 carrier frequency for
    % the frequency domain
    figure()
    subplot(2,1,1)
    plot(fA, amp2dB(abs(fftshift(plot1_5{i}))));
    title(['Final clean audio signal #' num2str(i)]);
    xlabel('Frequency (Hz)');
    ylabel('Magnitude (dB)');
    grid on;
```

```
    % Plot the final clean audio signal for each 5 carrier frequency for
    % the time domain
    subplot(2,1,2)
    plot(timeVector, plot1_6{i});
    title(['Final clean audio signal #' num2str(i)]);
    xlabel('Frequency (Hz)');
    ylabel('Magnitude (dB)');
    grid on;

end
```



Function definitions

```
% Converting amplitudes to decibels.
function AdB = amp2dB(A)

AdB = 20*log10(A + eps);

end

% A function for performing normalized fast fourier transforms.
function nfft = normalised_fft(signal)

nfft = fft(signal) / length(signal);

end

% Function for modulation of carrier frequencies using cos(2*pi*f*t).
function [c, demodulatedSignal, audioReceived, audioReceivedFreq] = ...
```

```
    carrierFreq(freq, timeVector, signal, cutoffFreq, fs)

c = cos(2*pi*freq*timeVector);
demodulatedSignal = signal.* c;
audioReceived = lowpass(demodulatedSignal, cutoffFreq, fs, ...
    'Steepness',0.99);
audioReceived = audioReceived - mean(audioReceived);
audioReceivedFreq = normalised_fft(audioReceived);

end

% Function for finding single-tone frequencies

function [Cleaned_Signal_f, cleaned_signal_t] = ...
    removetones(input, fA, initial_f, LPFcutoff, fs)

    % input must be shifted for following arithmetic to function correctly
    fftshifted = fftshift(input);

    % the threshold is set to a ridiculous 500 times the mean value.
    % any frequency above this value will be set to 0.
    threshold = 500*mean(abs(fftshifted));

    % for the length of the supplied frequency vector
    for i = 1:length(fA)
        % if the absolute value of the indexed frequency is greater than
        % the supplied initial frequency
        if abs(fA(i)) > initial_f
            % AND if that indexed value is greater than the threshold
            % Remove the frequency corresponding to that index
            if abs(fftshifted(i)) > threshold
                removefreq = i;
                % disp(i); % debug display
                % (uncomment to see indices and verify correct frequencies)
                fftshifted(removefreq) = 0;
                % disp(fftshifted(removefreq)); % also a debug
            end
        end
    end
    cleaned_signal_t = ...
        ifft(ifftshift(fftshifted)) * length(input);
    cleaned_signal_t = ...
        lowpass(cleaned_signal_t, LPFcutoff,fs, "Steepness", 0.99);
    Cleaned_Signal_f = ...
        normalised_fft(cleaned_signal_t);
end
```

Published with MATLAB® R2026a